

PUC Minas
Engenharia de Software

Análise de Eficácia de Testes com Teste de Mutação

Disciplina: Teste de Software

Paulo Henrique Fonseca de Assis
Matrícula: 829040

Belo Horizonte
2026

1. Preparação e análise inicial

Neste trabalho, analisei uma biblioteca com 50 operações matemáticas. Como meu repositório tinha apenas um README, utilizei uma cópia do projeto base indicado no enunciado. Mantive uma versão inicial na pasta baseline para conseguir repetir a comparação. Também ajustei o tipo de módulo para CommonJS, pois o código utilizava `require` e `module.exports`, e retirei os comentários sem mudar os cálculos.

Métrica	Inicial	Só testes	Final
Quantidade de testes	50	89	89
Cobertura de instruções	85,41%	100%	100%
Cobertura de branches	58,82%	100%	100%
Cobertura de funções	100%	100%	100%
Cobertura de linhas	98,64%	100%	100%
Mutation Score	73,71%	96,71%	99,01%

Todos os testes iniciais passaram. Apesar de quase todas as linhas terem sido executadas, o Stryker gerou 213 mutantes: 154 mortos, 3 com timeout, 44 sobreviventes e 12 sem cobertura. Isso mostrou que executar uma linha não significa conferir todos os comportamentos que ela pode ter. A cobertura de branches, de 58,82%, já indicava cenários que estavam faltando.

2. Mutante crítico: comparação no limite



```
94 function tanrenneitParalelsius(tanrenneit) { return (tanrenneit - 32) * 5/9; } ●●
95 function inverso(n) {
96   if (n === 0) throw new Error('Não é possível inverter o número zero. '); ●●
97   return 1 / n;
98 }
99
100
101 function areaCirculo(raio) { return Math.PI * Math.pow(raio, 2); }
102 function areaRetangulo(base, altura) { return base * altura; }
103 function perimetroRetangulo(base, altura) { return 2 * (base + altura); }
104 - function isMaiorQue(a, b) { return a > b; } ●▼
+ function isMaiorQue(a, b) { return a >= b; }
105 function isMenorQue(a, b) { return a < b; } ●●
106 function isEqual(a, b) { return a === b; } ●
107 function medianaArray(numeros) {
108   if (numeros.length === 0) throw new Error('Array vazio he possui mediana. '); ●●
109   const sorted = [...numeros].sort((a, b) => a - b); ●●●
110   const mid = Math.floor(sorted.length / 2);
111   if (sorted.length % 2 === 0) { ●●●
```

EqualityOperator Survived (104:36) More

Covered by 1 test (yet still survived)

O mutante 183 trocou `a > b` por `a >= b` em `isMaiorQue`. O teste original verificava apenas 10 e 5, que retornam `true` nos dois casos. Adicionei a comparação entre 5 e 5, esperando `false`, além dos cenários com valores maiores e menores. Assim, a troca do operador passou a ser detectada. O mutante ficou com status `Killed` na análise final.

3. Mutante crítico: conversão de temperatura



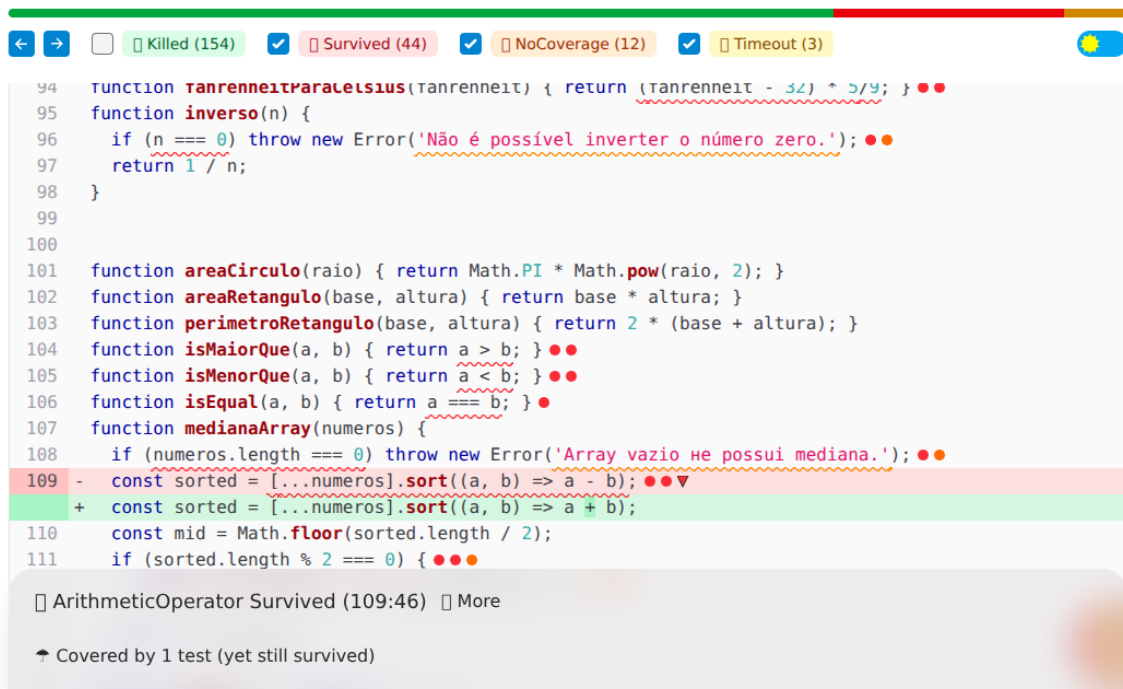
```
94 - function fahrenheitParaCelsius(fahrenheit) { return (fahrenheit - 32) * 5/9; }
95 + function fahrenheitParaCelsius(fahrenheit) { return (fahrenheit - 32) * 5 * 9; }
96
97 function inverso(n) {
98   if (n === 0) throw new Error('Não é possível inverter o número zero. ');
99   return 1 / n;
100 }
101
102 function areaCirculo(raio) { return Math.PI * Math.pow(raio, 2); }
103 function areaRetangulo(base, altura) { return base * altura; }
104 function perimetroRetangulo(base, altura) { return 2 * (base + altura); }
105 function isMaiorQue(a, b) { return a > b; }
106 function isMenorQue(a, b) { return a < b; }
107 function isEqual(a, b) { return a === b; }
108 function medianaArray(numeros) {
109   if (numeros.length === 0) throw new Error('Array vazio He possui mediana. ');
110   const sorted = [...numeros].sort((a, b) => a - b);
111   const mid = Math.floor(sorted.length / 2);
112   if (sorted.length % 2 === 0) {
113     return (sorted[mid] + sorted[mid - 1]) / 2;
114   }
115   return sorted[mid];
116 }
```

ArithmeticOperator Survived (94:53) More

Covered by 1 test (yet still survived)

O mutante 163 alterou $(\text{fahrenheit} - 32) * 5 / 9$ para $(\text{fahrenheit} - 32) * 5 * 9$. O teste original usava somente 32°F. Como a subtração resulta em zero, a conversão continuava retornando zero mesmo com o erro. Adicionei 212°F, 77°F e -40°F, esperando 100°C, 25°C e -40°C. Também verifiquei a conversão de Celsius para Fahrenheit. O mutante passou a ser morto.

4. Mutante crítico: ordenação da mediana



```
94 function fahrenheitParaCelsius(fahrenheit) { return (fahrenheit - 32) * 5/9; }
95 function inverso(n) {
96   if (n === 0) throw new Error('Não é possível inverter o número zero. ');
97   return 1 / n;
98 }
99
100 function areaCirculo(raio) { return Math.PI * Math.pow(raio, 2); }
101 function areaRetangulo(base, altura) { return base * altura; }
102 function perimetroRetangulo(base, altura) { return 2 * (base + altura); }
103 function isMaiorQue(a, b) { return a > b; }
104 function isMenorQue(a, b) { return a < b; }
105 function isEqual(a, b) { return a === b; }
106 function medianaArray(numeros) {
107   if (numeros.length === 0) throw new Error('Array vazio He possui mediana. ');
108   const sorted = [...numeros].sort((a, b) => a - b);
109 - const sorted = [...numeros].sort((a, b) => a - b);
110 + const sorted = [...numeros].sort((a, b) => a + b);
111   const mid = Math.floor(sorted.length / 2);
112   if (sorted.length % 2 === 0) {
113     return (sorted[mid] + sorted[mid - 1]) / 2;
114   }
115   return sorted[mid];
116 }
```

ArithmeticOperator Survived (109:46) More

Covered by 1 test (yet still survived)

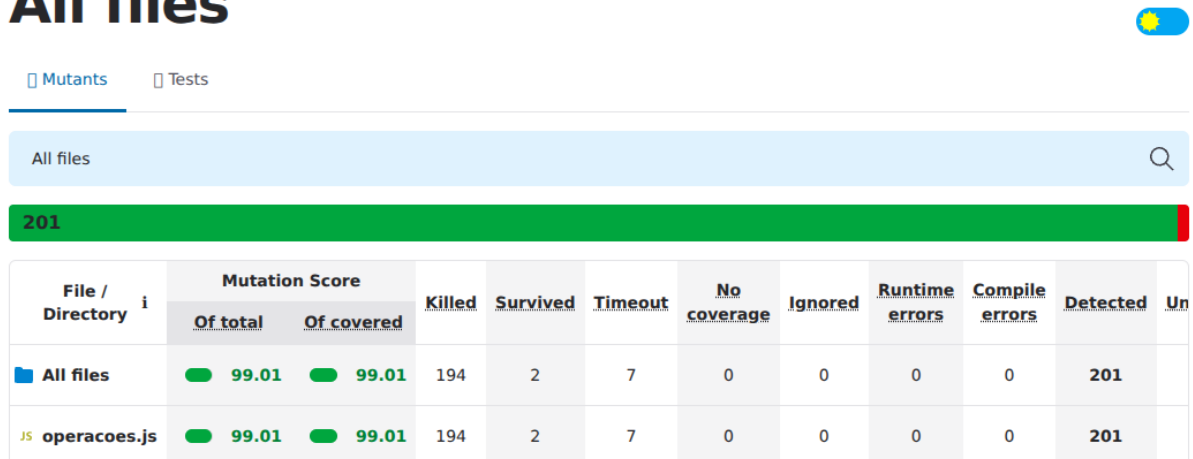
O mutante 203 trocou $a - b$ por $a + b$ no comparador de ordenação da mediana. O teste original recebia [1, 2, 3, 4, 5], que já estava ordenado e não expunha bem o erro. Adicionei [20, 1, 10, 2], cuja mediana é 6, e [30, 2, 10], cuja mediana é 10. Também conferi que a função não altera o array de entrada. Esses casos detectaram o mutante, que terminou como Killed.

5. Solução implementada e resultados finais

Adicionei 39 casos, chegando a 89 testes passando. Além dos três exemplos, incluí arrays vazios e unitários, números negativos, mensagens de erro, fatorial de zero e um, números compostos, Fibonacci nos primeiros termos e limites do clamp. Usei StrykerJS 10.0.0 com Jest 29.7.0, análise de cobertura por teste e todos os tipos de mutação habilitados para o arquivo de operações.

Somente com os novos testes, o score aumentou de 73,71% para 96,71%, mantendo os mesmos 213 mutantes. Nesse ponto, restaram sete sobreviventes equivalentes. Para simplificar o código, removi o retorno antecipado do fatorial de zero/um, pois o laço já devolvia 1, e o retorno do produto de array vazio, pois o reduce já tinha valor inicial 1. Essa etapa reduziu o total para 203 mutantes. Portanto, o resultado final também depende de uma simplificação do código; não atribuí todo esse ganho aos testes.

All files



The screenshot shows the StrykerJS report interface. At the top, there's a toggle for 'Mutants' and 'Tests'. Below it, a search bar contains 'All files'. A green progress bar indicates 201 mutants. The main table shows the following data:

File / Directory	Mutation Score		Killed	Survived	Timeout	No coverage	Ignored	Runtime errors	Compile errors	Detected	Un
	Of total	Of covered									
All files	99.01	99.01	194	2	7	0	0	0	0	201	
JS operacoes.js	99.01	99.01	194	2	7	0	0	0	0	201	



O resultado final foi 99,01%: 194 mortos, 7 timeouts, 2 sobreviventes e nenhum mutante sem cobertura. O cálculo é $(194 + 7) / 203 \times 100$. Os timeouts contam como detectados no Stryker, mas não significam que uma assertão falhou. Os dois sobreviventes trocam `<` por `<=` e `>` por `>=` no clamp. Para entradas numéricas, quando o valor é igual ao limite, retornar o limite ou o próprio valor produz o mesmo resultado. Nenhum mutante foi excluído para elevar o score.

6. Conclusão

Com a análise, percebi que uma cobertura alta pode esconder testes pouco específicos. Os casos de igualdade, entradas desordenadas e valores diferentes de zero fizeram diferença porque verificam situações em que um erro realmente muda a resposta. O teste de mutação ajudou a encontrar essas lacunas, mas também exigiu analisar os sobreviventes equivalentes. A meta de mais de 98% foi atingida, sem que isso signifique que o programa esteja livre de defeitos.

Repositório: <https://github.com/Paulooassis/Teste-de-muta-o>

Base: <https://github.com/CleitonSilvaT/operacoes-mutante>

Evidências: reports/initial, reports/tests-only e reports/final. A cópia preserva a origem, mas não é um fork registrado no GitHub. As alterações estão preparadas localmente; o envio remoto depende de autorização.